

BATTERY MANAGEMENT SYSTEM

High-Level Design Document

Document ID: HLD-BMS-001

Revision: 1.1

Status: Approved for Engineering Analysis

Classification: Synthetic Demonstration Data

1. System Overview

The Battery Management System coordinates its functions through typed AUTOSAR interfaces and ports. Software components exchange signals through provider and consumer ports.

2. Software Architecture

The primary software components are BatteryManager, CellMonitorControl, ChargeControl, ContactorControl, and ThermalManager.

3. Software Components

Component	Role	Primary Responsibility
BatteryManager	Coordinator	Supervises pack state and limits.
CellMonitorControl	Provider	Measures cell voltages and temperatures.
ChargeControl	Consumer	Regulates charge current.
ContactorControl	Consumer	Opens and closes the high-voltage contactors.
ThermalManager	Consumer	Controls pack cooling.

4. Interfaces

Interface	Provider	Consumer	Signals
ICellVoltage	CellMonitorControl	BatteryManager	CellVoltage
ICellTemperature	CellMonitorControl	ThermalManager	CellTemperature
IChargeLimit	BatteryManager	ChargeControl	ChargeCurrentLimit
IContactorCommand	BatteryManager	ContactorControl	ContactorCommand
IPackStatus	BatteryManager	ThermalManager	PackStatus

5. Ports

Component	Port	Direction	Interface
CellMonitorControl	CellVoltage_Out	P-PORT	ICellVoltage
BatteryManager	CellVoltage_In	P-PORT	ICellVoltage
CellMonitorControl	CellTemperature_Out	P-PORT	ICellTemperature
ThermalManager	CellTemperature_In	R-PORT	ICellTemperature
BatteryManager	ChargeLimit_Out	P-PORT	IChargeLimitLegacy
ChargeControl	ChargeLimit_In	R-PORT	IChargeLimit
BatteryManager	ContactorCommand_Out	P-PORT	IContactorCommand
ContactorControl	ContactorCommand_In	R-PORT	IContactorCommand
BatteryManager	PackStatus_Out	P-PORT	IPackStatus
ThermalManager	PackStatus_In	R-PORT	IPackStatus

6. Signals

Signal	Data Type	Source	Destination
CellVoltage	uint16	CellMonitorControl	BatteryManager
CellTemperature	int16	CellMonitorControl	ThermalManager
ChargeCurrentLimit	uint16	BatteryManager	ChargeControl
ContactorCommand	uint8	BatteryManager	CellMonitorControl
PackStatus	uint8	BatteryManager	ThermalManager

7. Dependencies

Source	Relationship	Target	Reason
CellMonitorControl	PROVIDES_INTERFACE	ICellVoltage	Publishes cell voltage.
BatteryManager	REQUIRES_INTERFACE	ICellVoltage	Consumes cell voltage.
CellMonitorControl	PROVIDES_INTERFACE	ICellTemperature	Publishes cell temperature.
BatteryManager	PROVIDES_INTERFACE	IChargeLimit	Publishes charge current limit.
ChargeControl	REQUIRES_INTERFACE	IChargeLimit	Consumes charge current limit.
BatteryManager	PROVIDES_INTERFACE	IContactorCommand	Publishes contactor command.
ContactorControl	REQUIRES_INTERFACE	IContactorCommand	Consumes contactor command.
BatteryManager	PROVIDES_INTERFACE	IPackStatus	Publishes pack status.
ThermalManager	REQUIRES_INTERFACE	IPackStatus	Consumes pack status.

8. Functional Flows

Flow F-001: Cell Voltage Distribution

CellMonitorControl publishes CellVoltage through ICellVoltage. BatteryManager receives the signal through CellVoltage_In and updates its internal state model.

Flow F-002: Cell Temperature Distribution

CellMonitorControl publishes CellTemperature through ICellTemperature. ThermalManager receives the signal through CellTemperature_In and updates its internal state model.

Flow F-003: Charge Current Limit Distribution

BatteryManager publishes ChargeCurrentLimit through IChargeLimit. ChargeControl receives the signal through ChargeLimit_In and updates its internal state model.

Flow F-004: Contactor Command Distribution

BatteryManager publishes ContactorCommand through IContactorCommand. ContactorControl receives the signal through ContactorCommand_In and updates its internal state model.

Flow F-005: Pack Status Distribution

BatteryManager publishes PackStatus through IPackStatus. ThermalManager receives the signal through PackStatus_In and updates its internal state model.

9. Integration Constraints

All provider and consumer ports shall reference compatible interfaces. Signal names shall remain consistent between interface definitions and functional flows. Architecture dependencies must be traceable to an approved source section.

10. Engineering Traceability

This synthetic HLD supports extraction, retrieval, validation and revision comparison experiments.